Naive Bayes Classifier

Abishek

11. November 2024

1. Introduction
2. The problem
3. Naive Bayes model using Python
4. Implementation
5. References

Introduction

What is Naive Bayes Classifier I

Naive Bayes classifier is a supervised-learning technique.

We have samples of data representative of different classes. Then using Bayes Theorem we calculate the probability of new data being in each class.

Why is it called Naïve Bayes?

- **Naïve**: Assumes **feature independence**; each feature independently contributes to classification.
  - Example: Identifies a fruit as an apple based on **color, shape**, and **taste**, assuming these features are independent.

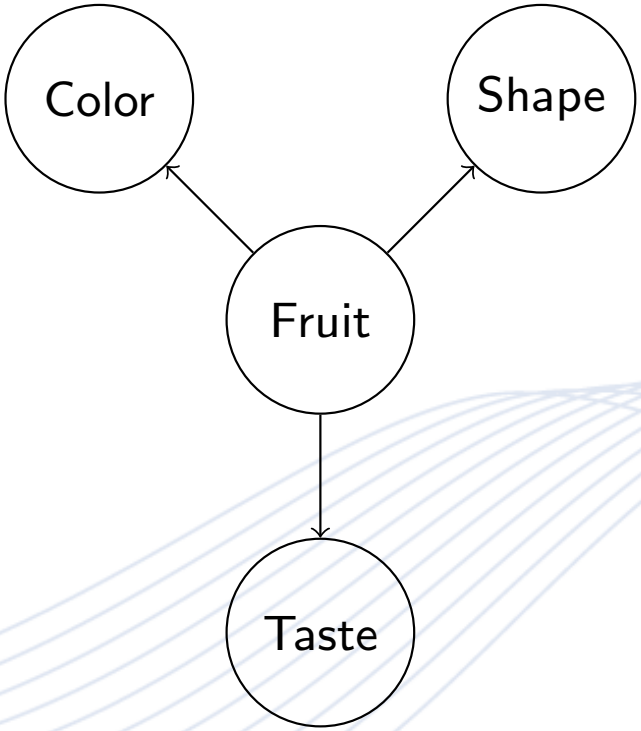

Abbildung: Naïve Bayes Classifier Example

- **Bayes**: It is called Bayes because it depends on the principle of Bayes' Theorem.

  Bayes' Theorem: Also known as **Bayes' Rule** or **Bayes' Law**, it is used to determine the probability of a hypothesis based on prior knowledge and conditional probability.

  The formula for Bayes' theorem is given by:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

  Where:

  - $P(A|B)$: **Posterior Probability** - The probability of the hypothesis A given the observed event B.
  - $P(B|A)$: **Likelihood Probability** - The probability of the evidence B given that hypothesis A is true.
  - $P(A)$: **Prior Probability** - The initial probability of the hypothesis A before observing any evidence.
  - $P(B)$: **Marginal Probability** - The probability of observing the evidence B.

When to use the Naïve Bayes Classifier algorithm:

- If you have a moderate or large training data set.
- If the instances have several attributes.
- Given the classification parameter, attributes which describe the instances should be conditionally independent.

Types of Naïve Bayes Algorithms

Naïve Bayes has **3 main types**:

- **Gaussian Naïve Bayes**
- **Multinomial Naïve Bayes**
- **Bernoulli Naïve Bayes**

1. Gaussian Naïve Bayes

Description:

- Suitable for **continuous data**.
- Assumes data in each class follows a **Gaussian (Normal) distribution**.
- Requires calculation of **mean** (μ) and **variance** (σ^2) for each class.

Formula:

$$P(x|\text{Class}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Applications:

- **Medical Diagnosis**: Predict diseases based on continuous indicators (e.g., blood pressure).
- **Iris Classification**: Classify flower species based on features like petal length.

2. Multinomial Naïve Bayes

Description:

- Designed for data that is **multinomially distributed** (e.g., frequency data).
- Suitable for **count-based data** such as word counts in text classification.

Mechanism:

- Treats feature vectors as **event frequencies** (e.g., word counts).
- Assumes each feature has a probability of occurrence.

Applications:

- **Text Classification**: Document categorization, spam detection, sentiment analysis.
- **Product Categorization**: Classify products based on descriptions by term frequencies.

3. Bernoulli Naïve Bayes

Description:

- Designed for **binary data** (0 or 1 values).
- Treats features as **boolean variables** representing presence or absence.

Mechanism:

- Considers whether a feature occurs (1) or does not occur (0).

Applications:

- **Document Classification**: Identify documents based on keyword presence (e.g., spam detection).
- **Image Recognition**: Classify binary images based on presence of specific features.

The problem

Naive Bayes I

Think about targeted marketing:

- Users in specific age ranges with interest in certain products
- Users with specific income levels that indicate purchasing power
- For-profit organizations using data to predict customer buying behavior
- ...

$\Rightarrow$ Need of a predictive model for user behavior

Naive Bayes model using Python

Dataset Introduction

Objective: Predict whether a user will make a purchase based on their age and salary.
Features:

- **Age**: User's age in years (numerical).
- **Salary**: User's annual salary (numerical).
- **Purchase (Target Variable)**: 1 (purchase), 0 (no purchase).

Example Data Entries

Age	Salary	Purchase
22	45,000	0
34	60,000	1
45	80,000	1
29	50,000	0
52	120,000	1

Objective

Goal: Use age and salary data to predict whether a user will make a purchase.
Model: Naïve Bayes classifier, assuming age and salary are independent in predicting the likelihood of a purchase.

Naïve Bayes Formula

The Naïve Bayes model is based on Bayes' theorem:

$$P(B|X) = \frac{P(X|B) \cdot P(B)}{P(X)}$$

- $P(B|X)$: Probability of purchase, given Age and Salary.
- $P(X|B)$: Probability of observing Age and Salary, given that the user purchased the product.
- $P(B)$: Prior probability of purchase.
- $P(X)$: Probability of observing Age and Salary.

Independence Assumption

Naïve Bayes assumes independence between Age and Salary:

$$P(X|B) = P(\text{Age}|B) \times P(\text{Salary}|B)$$

Gaussian Naïve Bayes

For continuous features, Gaussian Naïve Bayes assumes a normal distribution:

$$P(x|B) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

where:

- μ is the mean.
- σ^2 is the variance for each feature (Age or Salary).

Implementation

Packages I

For building this model, we leverage several key Python libraries to handle data manipulation, analysis, model building, and evaluation for predicting purchase behavior.

- **Pandas Purpose:** Powerful library for data manipulation and analysis.
  Usage
  - Read, inspect, clean, and preprocess the dataset.
  - Load data into DataFrame, handle missing values, filter, sort, and perform statistical analysis.
- **NumPy Purpose:** Supports high-performance mathematical and statistical operations on large arrays.
  Usage:
  - Perform numerical calculations for probability estimation.
  - Handle feature transformations and vectorized calculations.
- **Scikit-Learn (sklearn) Purpose:** Machine learning library for data preprocessing, model building, evaluation, and tuning.
  Usage:
  - **Train-Test Split:** Split dataset into training/testing subsets.

- **Naïve Bayes Model:** Use GaussianNB for continuous features (age, salary).
- **Performance Metrics:** Evaluate using accuracy, precision, recall, and confusion matrix .

- **Matplotlib and Seaborn Purpose:** Popular libraries for data visualization.
  Usage:
  - **Matplotlib:** Create histograms and scatter plots for feature distributions.
  - **Seaborn:** Create heatmaps and confusion matrix visualizations.

- **re (Regular Expressions) Purpose:** Allows text manipulation and pattern extraction. **Usage:**
  - Clean or standardize text-based features (e.g., categorical data).
  - Extract patterns if the dataset has textual features.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
```

Dataset I

We load the dataset from a CSV file:

- `X`: Contains the features (Age and Estimated Salary).
- `y`: Target variable (Purchased).

```
import pandas as pd

# Load dataset
dataset = pd.read_csv('SocialNetworkAds.csv')

# Extract features (Age and Estimated Salary) and target (Purchased)
X = dataset.iloc[:, [2, 3]].values  # Features
y = dataset.iloc[:, -1].values  # Target
```

Splitting the Dataset I

Next, we split the data into training and testing sets:

- 75% for training and 25% for testing.
- traintestsplit() ensures random splitting, allowing for unbiased evaluation.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
```

This ensures that 75% of the data is used for training and 25% for testing. The `train_test_split()` function randomly splits the data, allowing for unbiased evaluation.

Feature Scaling I

S

ince the features (Age and Estimated Salary) may have different scales, we scale them to ensure that the model performs optimally. We apply standard scaling to both training and testing data.

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Training the Naive Bayes Model I

We initialize a Gaussian Naive Bayes classifier and train it using the scaled training data (`Xtrain and ytrain`).

```
classifier = GaussianNB()
classifier.fit(X_train, y_train)
```

Prediction and Evaluation I

We use the predict function to make predictions on the test set .

```
y_pred = classifier.predict(X_test)
```

To evaluate the performance of the model, we:

- Use a confusion matrix to analyze the true positives, false positives, true negatives, and false negatives.
- Calculate the accuracy score, which indicates how well the model performs.

```
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("Accuracy:", accuracy)
```

Visualizing the Training Set Results I

To better understand the model's predictions, we visualize the decision boundary and the data points in the training set. The contourf function helps create a decision boundary, and the scatter function plots the actual data points.

```
X_set, y_set = X_train, y_train
X1, X2 = np.meshgrid(np.arange(start=X_set[:, 0].min() - 1, stop=X_set[:, 0].max() + 1, step
    =0.01),
np.arange(start=X_set[:, 1].min() - 1, stop=X_set[:, 1].max() + 1, step=0.01))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape),
alpha=0.75, cmap=ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
c=ListedColormap(('red', 'green'))(i), label=j)
plt.title('Naive Bayes (Training set)')
plt.xlabel('Age')
plt.ylabel('Estimated Salary')
plt.legend()
plt.show()
```

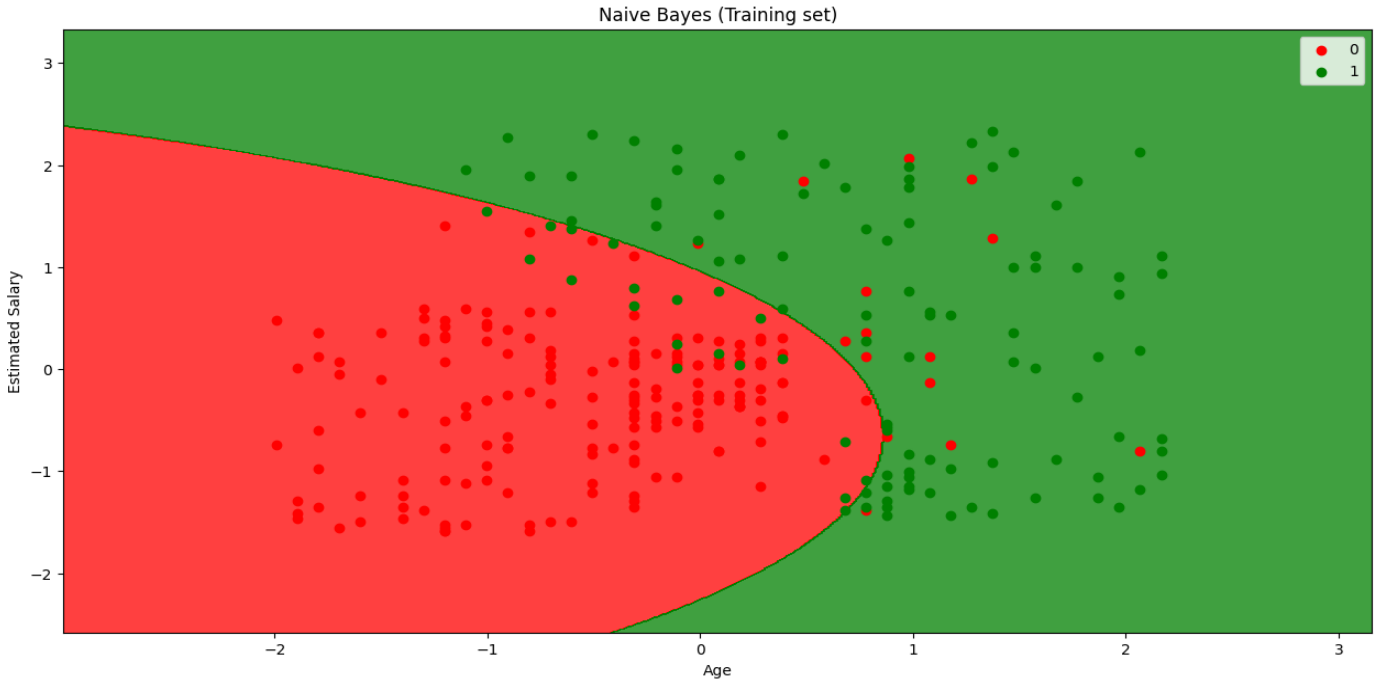

Abbildung: Training set

Visualizing the Test Set Results I

T

he same visualization approach is used to plot the decision boundary and data points for the test set.

```
X_set, y_set = X_test, y_test
X1, X2 = np.meshgrid(np.arange(start=X_set[:, 0].min() - 1, stop=X_set[:, 0].max() + 1, step
    =0.01),
np.arange(start=X_set[:, 1].min() - 1, stop=X_set[:, 1].max() + 1, step=0.01))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape),
alpha=0.75, cmap=ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
c=ListedColormap(('red', 'green'))(i), label=j)
plt.title('Naive Bayes (Test set)')
plt.xlabel('Age')
plt.ylabel('Estimated Salary')
plt.legend()
plt.show()
```

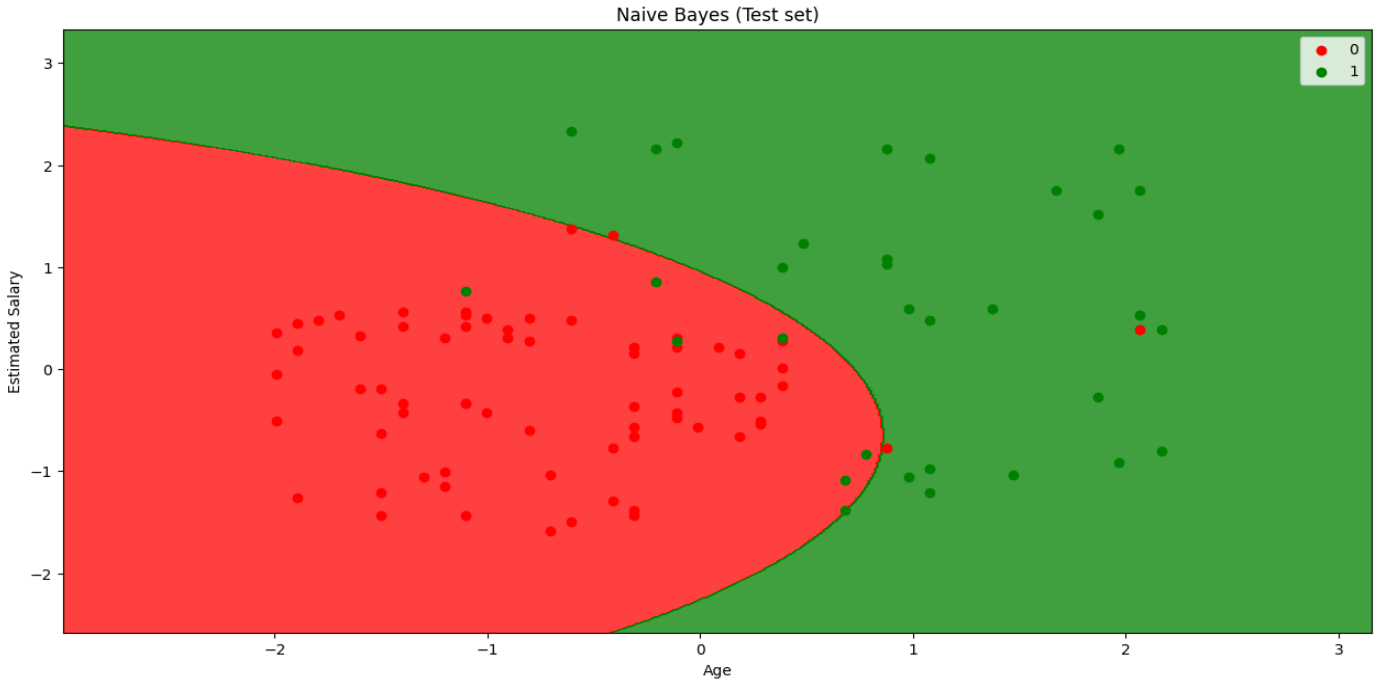

Abbildung: Test set

Naive Bayes Hyperparameters

- **Key Hyperparameters for Fine-Tuning**
  - **var_smoothing**: Controls smoothing applied to the variance to prevent instability.
  - **priors**: Sets prior probabilities for each class; useful for known or imbalanced class distributions.
  - **fit_prior**: Boolean flag to determine if priors are learned from the data or set manually.

Understanding the Impact of Each Hyperparameter

- **1. Variance Smoothing (`var-smoothing`)**
  - **Purpose**: Adds a small constant to feature variance to avoid division by zero.
  - **Effect**:
    - Higher values: More generalized model (better for noisy data).
    - Lower values: Potential overfitting due to sensitivity to small variance differences.
- **2. Class Prior Probabilities (`priors`)**
  - **Purpose**: Allows specifying likelihood of each class.
  - **Effect**: Useful for imbalanced datasets (e.g., more users who don't purchase).

- **3. Fit Priors (`fit-prior`)**
  - **Purpose**: Controls if priors are computed from data (`True`) or set manually.
  - **Effect**:
    - `True`: Model learns class priors from data.
    - `False`: Requires manual setting of `priors`.

```
classifier = GaussianNB(var-smoothing=1e-7)
classifier = GaussianNB(fit-prior=False)
classifier = GaussianNB(priors=[0.7, 0.3])
```

Limitations of Naive Bayes

- **Assumption of Feature Independence:**
  Naive Bayes assumes that features are conditionally independent, which is often unrealistic and can lead to oversimplified models.
- **Sensitivity to Irrelevant Features:**
  Irrelevant or redundant features can negatively impact performance, as Naive Bayes doesn't automatically downweight less important features.
- **Handling Imbalanced Data:**
  Naive Bayes may struggle with imbalanced datasets, often becoming biased toward the majority class.
- **Limited Handling of Continuous Features:**
  Gaussian Naive Bayes assumes continuous features follow a normal distribution, which may not suit all datasets well.
- **Limited Model Flexibility:**
  The simplicity of Naive Bayes limits its ability to model complex relationships, making it unsuitable for some non-linear problems.

Improvements for Naive Bayes

- **Feature Engineering and Selection:**
  Use techniques like TF-IDF, n-grams, and mutual information to select and engineer impactful features.
- **Advanced Smoothing Techniques:**
  Apply alternative smoothing (e.g., Lidstone or Good-Turing) and optimize smoothing parameters to improve accuracy.
- **Techniques for Imbalanced Data:**
  Adjust class priors, apply cost-sensitive Naive Bayes, or use SMOTE to balance the dataset.
- **Hybrid and Ensemble Approaches:**
  Combine with algorithms (e.g., SVM) or use ensemble methods like AdaBoost for better generalization and accuracy.
- **Improved Handling of Continuous Features:**
  Use kernel density estimation for non-Gaussian distributions or discretize continuous variables for simpler modeling.

Applications of Naive Bayes Classifier I

1. Sentiment Analysis: It is used at Facebook to analyse status updates expressing positive or negative emotions.

2. Document Categorization: Google uses document classification to index documents and find relevancy scores i.e. the PageRank. PageRank mechanism considers the pages marked as important in the databases that were parsed and classified using a document classification technique.

3. Naive Bayes Algorithm is also used for classifying news articles about Technology, Entertainment, Sports, Politics, etc.

4. Email Spam Filtering-Google Mail uses Naive Bayes algorithm to classify your emails as Spam or Not Spam.

Advantages of the Naive Bayes Classifier Algorithm I

1. Naive Bayes Classifier algorithm performs well when the input variables are categorical
2. A Naive Bayes classifier converges faster, requiring relatively little training data than other discriminative models like logistic regression, when the Naive Bayes conditional independence assumption holds.
3. With Naive Bayes Classifier algorithm, it is easier to predict class of the test data set. A good bet for multi class predictions as well.
4. Though it requires conditional independence assumption, Naive Bayes Classifier has presented good performance in various application domains.

References

References I